

E3 — Testing con pytest sobre el pipeline de ventas

Explicación de la práctica y de los conceptos teóricos para repasar de cara al examen.

1. Qué se ha hecho en la práctica

Objetivo: escribir **tests automáticos** que comprueben que el paquete `ventas_app` de la E2 funciona correctamente, tanto con datos pequeños inventados como con el CSV real del curso.

Estructura creada dentro de `1_programacion_avanzada_python/` :

```
1_programacion_avanzada_python/  
  Datos/  
  ventas_app/          # código que se prueba (E2)  
  tests/  
    test_validator.py   # tests del validador (fixture, parametrize, errores)  
    test_loader.py      # tests de la carga de ficheros  
    test_integration.py # tests con el ventas.csv real (marca integration)  
  pytest.ini           # configuración de pytest
```

Parte	Qué se hace	Herramienta clave
0. Calentamiento	Ejecutar los tests de ejemplo del profesor y romper uno a propósito para ver cómo informa pytest del fallo	<code>pytest -q</code>
1. Fixture	Datos de prueba en memoria (3 filas: 1 válida, 2 inválidas) y tests del número de válidas/inválidas	<code>@pytest.fixture</code>
2. Errores y bordes	Error si el fichero no existe y precios límite <code>0</code> , <code>-1</code> , <code>1</code>	<code>pytest.raises</code> , <code>@pytest.mark.parametrize</code>
3. Integración	Tests con el <code>ventas.csv</code> real: 140 válidas y 10 inválidas	<code>@pytest.mark.integration</code> , <code>-m</code>
4. Documentar	Comandos de test en el README	—

Resultado:

```
pytest -q                -> 14 passed  
pytest -q -m "not integration" -> 12 passed, 2 deselected
```

Hay 14 tests en total (el enunciado pide al menos 8), de los cuales 2 son de integración.

Tests escritos:

test_validator.py

Test	Qué comprueba
test_mini_tiene_una_fila_valida	La fixture produce 1 fila válida
test_mini_tiene_dos_filas_invalidas	La fixture produce 2 filas inválidas
test_importe_se_calcula_en_filas_validas	importe = 2 × 10.0 = 20.0
test_validacion_no_modifica_el_dataframe_original	El <code>.copy()</code> protege el DataFrame original
test_precio_limite (×3)	Precio 0 → inválido, -1 → inválido, 1 → válido
test_unidades_texto_es_invalida	"na" en unidades hace la fila inválida
test_faltan_columnas_lanza_validation_error	Si falta una columna → <code>ValidationError</code>

test_loader.py

Test	Qué comprueba
test_load_path_no_existe_lanza_error	Fichero inexistente → <code>DataLoadError</code>
test_load_formato_no_soportado_lanza_error	Extensión <code>.txt</code> → <code>DataLoadError</code>
test_load_csv_devuelve_dataframe	Un CSV correcto se carga con sus filas

test_integration.py

Test	Qué comprueba
test_csv_curso_tiene_140_validas	CSV real → 140 válidas
test_csv_curso_tiene_10_invalidas	CSV real → 10 inválidas

Nota: el enunciado llama al test de integración `test_csv_curso_tiene_ocho_validas`, pero el CSV del curso da 140 filas válidas (igual que el checkpoint de la E1). Por eso se ha nombrado según el resultado real.

2. Conceptos teóricos

2.1 ¿Para qué sirven los tests?

- Un **test** es un código que ejecuta otro código y **comprueba automáticamente** que el resultado es el esperado.
- **Detectan errores al cambiar el código (regresiones).** Si al refactorizar algo se rompe, el test falla y avisa en el momento.
- **Documentan el comportamiento:** leyendo los tests se sabe qué debe hacer cada función, por ejemplo que "precio 0 es inválido".

- **Dan confianza para modificar el código:** si los tests siguen en verde, el cambio no ha roto nada.
- **Se ejecutan en segundos,** frente a comprobar a mano ejecutando el programa y mirando la salida.

2.2 Tipos de tests

Tipo	Qué prueba	Ejemplo en la práctica	Velocidad
Unitario	Una función aislada, con datos pequeños controlados	<code>validar_ventas</code> con la fixture de 3 filas	Muy rápido
Integración	Varias piezas juntas con datos o recursos reales	<code>load + validar_ventas</code> con el <code>ventas.csv</code> real	Más lento
End-to-end	El sistema completo, como lo usaría el usuario	Ejecutar el CLI y revisar los ficheros generados	El más lento

Pirámide de tests: debe haber **muchos** tests unitarios, **algunos** de integración y **pocos** end-to-end. Los unitarios son rápidos y localizan el error exacto; los de integración comprueban que las piezas encajan.

2.3 Cómo encuentra pytest los tests (convenciones)

- Busca ficheros que se llamen `test_*.py` (o `*_test.py`).
- Dentro, ejecuta las funciones cuyo nombre empieza por `test_`.
- **No hace falta registrar nada:** basta con respetar los nombres.
- Cada test es independiente: el fallo de uno no afecta a los demás.

2.4 `assert`

```
def test_mini_tiene_una_fila_valida(ventas_mini):
    validos, _ = validar_ventas(ventas_mini)
    assert len(validos) == 1
```

- `assert condición` no hace nada si la condición es `True`, y hace **fallar el test** si es `False`.
- pytest "reescribe" los `assert` para que, cuando fallan, muestren **los valores reales**. Por ejemplo, `assert 141 == 140` en lugar de un simple "falso".
- El `_` se usa para ignorar un valor devuelto que no interesa en ese test (aquí, `errores`).

Patrón AAA (Arrange - Act - Assert), la estructura típica de un test:

1. **Arrange (preparar):** crear los datos de entrada (la fixture o `crear_venta(...)`).
2. **Act (actuar):** llamar a la función que se prueba (`validar_ventas(...)`).
3. **Assert (comprobar):** verificar el resultado (`assert len(validos) == 1`).

2.5 Leer la salida de pytest

Símbolo	Significado
.	Test pasado
F	Test fallido (un <code>assert</code> es falso)
E	Error (excepción inesperada, por ejemplo dentro de una fixture)
s	Test saltado (<i>skipped</i>)

Parte 0 de la práctica: se cambia a propósito el valor esperado de un test que funciona. Por ejemplo, en `test_integration.py`:

```
VALIDAS_ESPERADAS = 141  # el valor correcto es 140
```

Al ejecutar `pytest -q` aparece una **F** y un informe como este:

```
>      assert len(validos) == VALIDAS_ESPERADAS
E      assert 140 == 141
E      + where 140 = len(...)
FAILED tests/test_integration.py::test_csv_curso_tiene_140_validas - assert 140 ==
141
1 failed, 13 passed
```

Este informe indica **qué test** falla, **en qué línea**, **qué valor se obtuvo** (140) y **cuál se esperaba** (141). El objetivo es aprender a leer este informe, que es lo que se usa para localizar los errores reales. Después hay que dejar el valor correcto.

2.6 Fixtures

```
@pytest.fixture
def ventas_mini():
    return pd.DataFrame({...})

def test_mini_tiene_una_fila_valida(ventas_mini):  # pytest la inyecta por nombre
    ...
```

- Una **fixture** es una función que prepara datos o recursos para los tests.
- Para usarla, basta con poner **su nombre como parámetro** del test. pytest la ejecuta y pasa el resultado automáticamente (esto se llama *inyección de dependencias*).
- Se ejecuta **de nuevo para cada test**, así que cada test recibe datos "limpios", aunque otro test los haya modificado.
- **Ventaja:** evita repetir el mismo código de preparación en cada test.
- **Datos en memoria:** la fixture crea el DataFrame directamente, sin leer ficheros. Así los tests son rápidos y controlados: se sabe exactamente qué filas son válidas.

Fixtures que trae pytest: `tmp_path` crea una **carpeta temporal** que se borra al terminar el test. Se usa para crear ficheros de prueba sin ensuciar el proyecto:

```
def test_load_formato_no_soportado_lanza_error(tmp_path):
    fichero = tmp_path / "ventas.txt"
    fichero.write_text("contenido")
    with pytest.raises(DataLoadError):
        load(fichero)
```

Fixture o función auxiliar: en `test_validator.py` también hay una función normal, `crear_venta(unidades, precio)`. Se usa cuando cada test necesita **datos distintos** (una fixture devuelve siempre lo mismo).

2.7 `pytest.raises` : comprobar que se lanza un error

```
def test_load_path_no_existe_lanza_error():
    with pytest.raises(DataLoadError):
        load("no_existe.csv")
```

- El test **pasa** si dentro del bloque `with` se lanza esa excepción.
- El test **falla** si no se lanza ninguna excepción, o si se lanza otra distinta.
- Sirve para probar que el código **falla bien**, con el error que corresponde, en situaciones incorrectas.

2.8 `@pytest.mark.parametrize` : el mismo test con varios datos

```
@pytest.mark.parametrize("precio, validas_esperadas", [
    (0, 0),
    (-1, 0),
    (1, 1),
])
def test_precio_limite(precio, validas_esperadas):
    validos, _ = validar_ventas(crear_venta(unidades=1, precio=precio))
    assert len(validos) == validas_esperadas
```

- Ejecuta el test **una vez por cada tupla** de la lista. Aquí son 3 tests distintos, y pytest los cuenta por separado.
- El primer argumento es un string con los nombres de los parámetros, separados por comas.
- Evita copiar y pegar el mismo test cambiando solo los datos.

Valores límite (edge cases): los errores suelen aparecer en los bordes de una regla. Con la regla `precio > 0`, hay que probar justo el borde (`0`), justo fuera (`-1`) y justo dentro (`1`). Probar solo con `10` no detectaría que alguien escribió `>=` en vez de `>`.

2.9 Marcas (*markers*) y el filtro `-m`

```
@pytest.mark.integration
def test_csv_curso_tiene_140_validas():
    ...
```

- Una **marca** es una etiqueta para clasificar tests.
- Se filtran al ejecutar con `-m`:

Comando	Qué ejecuta
<code>pytest -q</code>	Todos los tests
<code>pytest -q -m integration</code>	Solo los marcados como <code>integration</code>
<code>pytest -q -m "not integration"</code>	Todos menos los de integración

- **Para qué sirve:** durante el desarrollo se ejecutan los unitarios, que son rápidos, y los lentos o los que dependen de ficheros reales se reservan para momentos concretos (antes de entregar, en CI...).
- Las marcas propias **hay que registrarlas** en `pytest.ini`; si no, pytest muestra un aviso (`PytestUnknownMarkWarning`).
- Marcas que trae pytest: `skip` (saltar un test), `skipif` (saltarlo si se cumple una condición), `xfail` (se espera que falle) y `parametrize`.

2.10 `pytest.ini` : configuración

```
[pytest]
pythonpath = .
testpaths = tests
markers =
    integration: tests que leen ficheros reales del curso (más lentos)
```

Opción	Qué hace
<code>pythonpath = .</code>	Añade la carpeta actual al path, para que los tests puedan hacer <code>from ventas_app import ...</code>
<code>testpaths = tests</code>	Indica a pytest que busque los tests en <code>tests/</code>
<code>markers</code>	Registra las marcas personalizadas

- `-q` (*quiet*) produce una salida resumida: un símbolo por test. `-v` (*verbose*) muestra el nombre de cada test.

2.11 Por qué la arquitectura de la E2 facilita los tests

- En el script de la **E1**, todo se ejecutaba de arriba abajo nada más importarlo: cargaba el CSV, imprimía y escribía ficheros. **No se podía probar una parte sin ejecutar todo.**

- En la **E2**, cada responsabilidad está en una función de un módulo (`load` , `validar_ventas` ...). Eso permite:
- importar y probar **solo** la función que interesa;
- pasarle **datos inventados** (la fixture) en lugar del CSV real;
- comprobar los **errores propios** (`DataLoadError` , `ValidationError`) con `pytest.raises` .
- **Conclusión:** un código modular es un código testeable. Por eso la E2 va antes que la E3.

3. Comandos usados

```
pip install pytest                # instalar pytest en el entorno virtual
cd 1_programacion_avanzada_python
mkdir tests
touch pytest.ini tests/test_validator.py tests/test_loader.py tests/
test_integration.py
pytest -q                        # todos los tests
pytest -q -m "not integration"   # solo unitarios
pytest -v                        # salida detallada con el nombre de cada test
```

4. Preguntas típicas de examen (E3)

1. **¿Qué diferencia hay entre un test unitario y uno de integración?** El unitario prueba una función aislada con datos controlados; el de integración prueba varias piezas juntas con recursos reales (por ejemplo, el CSV del curso).
2. **¿Cómo encuentra pytest los tests?** Busca ficheros `test_*.py` y, dentro, funciones que empiezan por `test_` .
3. **¿Qué es una fixture y cómo se usa?** Una función con `@pytest.fixture` que prepara datos. Se usa poniendo su nombre como parámetro del test, y pytest la ejecuta y pasa el resultado.
4. **¿Qué ventaja tienen los datos en memoria frente a leer un fichero?** Los tests son más rápidos, no dependen de ficheros externos y se sabe exactamente qué resultado esperar.
5. **¿Para qué sirve `pytest.raises` ?** Para comprobar que el código lanza una excepción concreta. El test pasa si se lanza y falla si no.
6. **¿Para qué sirve `@pytest.mark.parametrize` ?** Para ejecutar el mismo test con distintos datos sin duplicar código. Cada combinación cuenta como un test.
7. **¿Por qué probar los precios 0, -1 y 1?** Porque son los valores límite de la regla `precio > 0` : el borde, justo fuera y justo dentro. Ahí es donde suelen estar los errores (por ejemplo, `>=` en vez de `>`).
8. **¿Para qué sirven las marcas y `-m` ?** Para clasificar tests y ejecutar solo una parte, como `-m "not integration"` para excluir los lentos.

9. **¿Por qué hay que registrar las marcas en `pytest.ini`?** Para que pytest las reconozca y no muestre avisos de marca desconocida.
10. **¿Qué es `tmp_path`?** Una fixture de pytest que crea una carpeta temporal para el test y la borra al terminar.
11. **¿Qué es el patrón AAA?** Arrange (preparar datos), Act (llamar a la función) y Assert (comprobar el resultado).
12. **¿Qué significan `.`, `F` y `E` en la salida de pytest?** Test pasado, test fallido (un assert es falso) y error inesperado, respectivamente.
13. **¿Por qué el código modular (E2) es más fácil de testear que el script (E1)?** Porque cada función se puede importar y probar por separado con datos inventados; el script ejecuta todo al importarlo.
14. **¿Qué es una regresión?** Un fallo que aparece en algo que antes funcionaba, normalmente tras un cambio. Los tests sirven para detectarlas en el momento.